

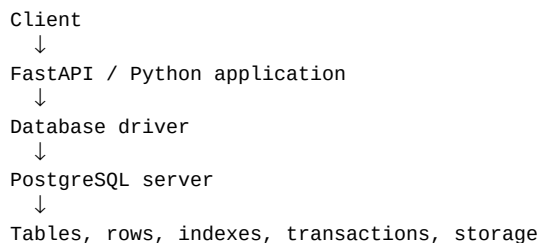
PostgreSQL — Detailed Beginner-to-Interview Textbook Notes

A practical guide from database fundamentals to Python, FastAPI, persistence, transactions, indexing, testing, and production engineering.

1. What Is PostgreSQL?

PostgreSQL is an open-source relational database management system (RDBMS). It is used to store, organize, query, update, and protect structured application data.

For a backend application, PostgreSQL is the persistent storage layer. Python/FastAPI handles application logic and HTTP requests, while PostgreSQL stores data so it survives application restarts.



PostgreSQL is not the same thing as SQL. SQL is the language used to communicate with relational databases. PostgreSQL is a database system that implements SQL and provides many additional capabilities.

2. Why Do We Need PostgreSQL?

If we store application data only in a Python dictionary, the data exists only inside the running Python process. If the application stops, the dictionary disappears.

```
documents = {
    "doc-001": {
        "name": "requirements.pdf",
        "status": "processed"
    }
}
```

PostgreSQL provides persistent storage, concurrent access, transactions, constraints, indexes, security, and reliable querying.

- Persistence: data remains after application restarts.
- Concurrency: multiple requests/users can access the database safely.
- Transactions: related changes can succeed or fail together.
- Constraints: the database can prevent invalid data.
- Indexes: frequently used queries can be accelerated.
- Querying: SQL provides filtering, joining, aggregation, and sorting.
- Reliability: PostgreSQL provides mature mechanisms for durability and recovery.

3. PostgreSQL Server Mental Model

A PostgreSQL deployment consists of a PostgreSQL server process and databases hosted by that server. Your application connects to the server using connection information such as host, port, database name, username, and password.

```
Host:      localhost
Port:      5432
Database:  ai_engineering
User:      postgres
Password:  <secret>
```

The default PostgreSQL TCP port is 5432. The application does not directly manipulate database files. It communicates with PostgreSQL through a database protocol and a client driver.

4. PostgreSQL Architecture

```
Python application
  ↓
Psycopg / database driver
  ↓
PostgreSQL connection
  ↓
SQL parser
  ↓
Query planner / optimizer
  ↓
Executor
  ↓
Tables / indexes / storage
```

4.1 Parser

The parser checks the SQL statement and turns it into an internal representation.

4.2 Planner / Optimizer

PostgreSQL determines how to execute the query efficiently. It can choose between sequential scans, index scans, different join strategies, and other execution plans.

4.3 Executor

The executor performs the selected plan and produces the result.

4.4 Storage

PostgreSQL stores persistent data and maintains indexes and transaction-related information.

5. Database → Schema → Table → Row → Column

```
PostgreSQL Server
  ■■■ Database
    ■■■ Schema
      ■■■ Table
        ■■■ Row
        ■■■ Row
        ■■■ Row
```

A database is a logical container for application data. A schema is a namespace inside a database. PostgreSQL commonly provides a public schema by default. A table stores related records. A row represents one record. A column represents one attribute of a record.

6. Creating a Database

```
CREATE DATABASE ai_engineering;
```

After connecting with psql, switch to the database with:

```
\c ai_engineering
```

7. PostgreSQL Data Types

- INTEGER: whole numbers.
- BIGINT: larger whole numbers.
- TEXT: variable-length text.
- VARCHAR: variable-length character data.
- BOOLEAN: true/false values.
- DATE: calendar dates.
- TIMESTAMP: timestamp without time-zone semantics.
- TIMESTAMPTZ: timestamp with time-zone-aware behavior.
- UUID: universally unique identifiers.

8. Creating the Documents Table

```
CREATE TABLE documents (  
    id TEXT PRIMARY KEY,  
    name TEXT NOT NULL,  
    document_type TEXT NOT NULL,  
    version TEXT,  
    status TEXT,  
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

This table gives the Document Service persistent storage for document metadata.

9. Constraints

Constraints allow the database to enforce rules about valid data.

9.1 PRIMARY KEY

A primary key uniquely identifies a row.

```
id TEXT PRIMARY KEY
```

9.2 NOT NULL

NOT NULL means a value must be provided.

```
name TEXT NOT NULL
```

9.3 UNIQUE

UNIQUE prevents duplicate values in a column or combination of columns.

```
email TEXT UNIQUE
```

9.4 CHECK

CHECK allows a database-level condition.

```
age INTEGER CHECK (age >= 0)
```

9.5 FOREIGN KEY

A foreign key represents a relationship between tables and prevents references to nonexistent parent records when the constraint is enforced.

10. CRUD in PostgreSQL

CRUD means Create, Read, Update, Delete.

10.1 INSERT — Create

```
INSERT INTO documents (  
    id, name, document_type, version, status  
)  
VALUES (  
    'doc-001', 'requirements.pdf', 'pdf', 'v1', 'uploaded'  
);
```

10.2 SELECT — Read

```
SELECT * FROM documents;  
  
SELECT id, name, status FROM documents;
```

10.3 UPDATE — Update

```
UPDATE documents  
SET status = 'processed'  
WHERE id = 'doc-001';
```

10.4 DELETE — Delete

```
DELETE FROM documents  
WHERE id = 'doc-001';
```

Always be careful with UPDATE and DELETE statements. Without a WHERE clause, they may affect every row.

11. Filtering with WHERE

```
SELECT *  
FROM documents  
WHERE status = 'processed';
```

Other common operators include =, !=, <>, >, <, >=, <=, IN, BETWEEN, LIKE, and IS NULL.

12. NULL

NULL does not mean zero, empty string, or false. It represents missing or unknown information.

```
SELECT *  
FROM documents  
WHERE version IS NULL;
```

Do not write version = NULL. SQL uses IS NULL and IS NOT NULL for NULL checks.

13. Sorting and Limiting

```
SELECT *  
FROM documents  
ORDER BY created_at DESC  
LIMIT 10;
```

ORDER BY determines result ordering. LIMIT restricts the number of returned rows.

14. Aggregation

Aggregate functions calculate values across multiple rows.

```
SELECT COUNT(*) FROM documents;
```

Other common aggregate functions include SUM, AVG, MIN, and MAX.

GROUP BY

```
SELECT status, COUNT(*)
FROM documents
GROUP BY status;
```

HAVING

```
SELECT status, COUNT(*)
FROM documents
GROUP BY status
HAVING COUNT(*) > 10;
```

WHERE filters rows before grouping. HAVING filters groups after aggregation.

15. Relationships Between Tables

- One-to-one: one record corresponds to one record.
- One-to-many: one parent has many child records.
- Many-to-many: many records relate to many other records through an association table.

For Project 1, a document may have many chunks, and a document may have many extracted artifacts or processing events.

16. JOIN

```
SELECT d.id, d.name, c.content
FROM documents d
JOIN document_chunks c
ON d.id = c.document_id;
```

Common join types include INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN.

17. SQL Categories

- DDL — Data Definition Language: CREATE, ALTER, DROP.
- DML — Data Manipulation Language: INSERT, UPDATE, DELETE.
- DQL — Data Query Language: SELECT.
- TCL — Transaction Control Language: COMMIT, ROLLBACK.

18. Transactions

```
BEGIN;

UPDATE documents
SET status = 'processing'
WHERE id = 'doc-001';
```

```
UPDATE documents
SET version = 'v2'
WHERE id = 'doc-001';

COMMIT;
```

If something goes wrong, the transaction can be rolled back.

```
ROLLBACK;
```

19. ACID

- Atomicity: a transaction is treated as a unit; its changes are committed together or rolled back.
- Consistency: committed transactions preserve database constraints and defined integrity rules.
- Isolation: concurrent transactions are managed according to PostgreSQL transaction isolation semantics.
- Durability: committed changes are designed to survive failures according to PostgreSQL durability mechanisms.

20. PostgreSQL from Python

Python needs a PostgreSQL client/driver to communicate with PostgreSQL. A practical choice is Psycopg 3.

```
pip install psycopg[binary]

import psycopg
```

21. Connecting Python to PostgreSQL

```
import psycopg

connection = psycopg.connect(
    "postgresql://user:password@localhost:5432/ai_engineering"
)
```

The connection string contains the database connection information. In real projects, credentials should not be hard-coded in source code.

22. Connection and Cursor

A connection represents communication with PostgreSQL. A cursor is used to execute SQL statements and retrieve results.

```
with psycopg.connect(DATABASE_URL) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT * FROM documents")
```

Context managers help ensure resources are properly managed.

23. SELECT from Python

```
import psycopg

with psycopg.connect(DATABASE_URL) as conn:
    with conn.cursor() as cur:
        cur.execute(
            "SELECT id, name, status FROM documents"
        )
        rows = cur.fetchall()
        for row in rows:
            print(row)
```

`fetchone()` returns one row, while `fetchall()` returns all available rows.

24. Parameterized Queries

Never build SQL by directly concatenating untrusted user input. Use parameterized queries instead.

```
cur.execute(
    "SELECT * FROM documents WHERE id = %s",
    ("doc-001",)
)
```

The driver safely handles the parameter. This is an important defense against SQL injection.

25. INSERT from Python

```
with psycopg.connect(DATABASE_URL) as conn:
    with conn.cursor() as cur:
        cur.execute(
            """
            INSERT INTO documents (
                id, name, document_type, version, status
            )
            VALUES (%s, %s, %s, %s, %s)
            """,
            (
                "doc-001",
                "requirements.pdf",
                "pdf",
                "v1",
                "uploaded",
            ),
        )
    conn.commit()
```

A transaction should be committed when the application intends to make the changes durable.

26. Commit and Rollback in Python

```
try:
    cur.execute(...)
    cur.execute(...)
    conn.commit()
except Exception:
    conn.rollback()
    raise
```

This pattern is important when multiple changes must succeed together.

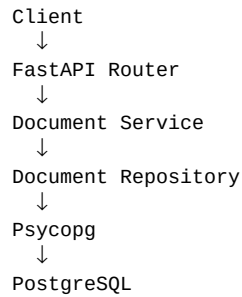
27. Connection Pooling

Opening a brand-new database connection for every HTTP request can be inefficient. Production applications commonly use connection pooling.

```
HTTP Request
↓
FastAPI
↓
Connection Pool
↓
Available PostgreSQL connection
↓
Query
↓
Return connection to pool
```

A pool allows connections to be reused while controlling the maximum number of active database connections.

28. FastAPI + PostgreSQL Architecture



Router handles HTTP request/response concerns. Service contains application/business logic. Repository contains persistence/database operations. Database stores persistent state.

29. Repository Pattern

```
class DocumentRepository:
    def create(self, document):
        ...

    def get(self, document_id):
        ...

    def list(self):
        ...

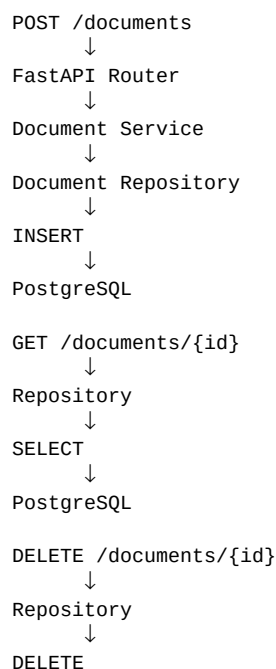
    def update(self, document_id, ...):
        ...

    def delete(self, document_id):
        ...
```

This keeps the API layer independent from low-level persistence details and makes testing and future changes easier.

30. Project 1 Document Service

The immediate Project 1 persistence milestone is to replace the in-memory Python dictionary with PostgreSQL.



↓
PostgreSQL

The API contract should remain clean while persistence moves behind the repository layer.

31. Configuration and Secrets

```
DATABASE_URL=postgresql://user:password@localhost:5432/ai_engineering
```

Use environment variables for local configuration and provide a safe .env.example. Do not commit real credentials or secrets to Git.

32. PostgreSQL with psql

```
psql -U postgres
\l
\c ai_engineering
\dt
\d documents
SELECT * FROM documents;
\q
```

- \l: list databases.
- \c: connect to a database.
- \dt: list tables.
- \d table_name: inspect table structure.
- \q: quit psql.

33. Migrations

A database schema changes over time. For example, you may initially create a documents table and later add columns, indexes, or relationships.

Do not rely on manually changing production databases. Use database migrations so schema changes are versioned and repeatable.

A migration system allows a team to answer: what schema exists, what changed, and in what order should changes be applied?

34. Indexes

```
CREATE INDEX idx_documents_status
ON documents(status);
```

An index can reduce the work required to find matching rows, but indexes are not free.

- Indexes consume storage.
- INSERT, UPDATE, and DELETE operations may become more expensive because indexes must be maintained.
- Too many indexes increase maintenance overhead.
- Indexes should be created based on actual query patterns.

35. EXPLAIN and EXPLAIN ANALYZE

```
EXPLAIN
SELECT *
```

```
FROM documents
WHERE status = 'processed';

EXPLAIN ANALYZE
SELECT *
FROM documents
WHERE status = 'processed';
```

EXPLAIN shows the plan PostgreSQL intends to use. EXPLAIN ANALYZE actually executes the query and reports runtime information. Do not assume an index or query is faster; measure the execution plan.

36. Query Execution Mental Model

SQL is declarative. You describe what data you want, and PostgreSQL determines an execution strategy.

```
SELECT name
FROM documents
WHERE status = 'processed'
ORDER BY created_at DESC
LIMIT 10;
```

A useful logical model is:

```
FROM
→ WHERE
→ GROUP BY
→ HAVING
→ SELECT
→ ORDER BY
→ LIMIT
```

The physical execution plan may differ from this logical order because PostgreSQL's optimizer can rearrange operations for efficiency.

37. DELETE vs TRUNCATE vs DROP

- DELETE: removes rows from a table and can use a WHERE clause.
- TRUNCATE: removes all rows efficiently from a table.
- DROP: removes the database object itself, such as a table.

```
DELETE FROM documents WHERE id = 'doc-001';
TRUNCATE TABLE documents;
DROP TABLE documents;
```

38. SQL Injection

SQL injection occurs when untrusted input changes the meaning of a SQL statement.

```
# Unsafe
query = "SELECT * FROM documents WHERE id = '" + document_id + "'"

# Safe
cur.execute(
    "SELECT * FROM documents WHERE id = %s",
    (document_id,)
)
```

Parameterized queries should be the default approach when values come from users or external systems.

39. PostgreSQL Testing

- Use a separate test database.
- Use fixtures for known test data.

- Keep tests isolated.
- Use rollback or cleanup strategies.
- Test successful queries.
- Test missing records.
- Test constraint violations.
- Test transaction failure behavior.
- Test connection/database failures.

Database code should be tested against a real PostgreSQL test database rather than relying only on mocks.

40. Example Repository Operation

```
class DocumentRepository:
    def __init__(self, database_url: str):
        self.database_url = database_url

    def get(self, document_id: str):
        with psycopgp.connect(self.database_url) as conn:
            with conn.cursor() as cur:
                cur.execute(
                    """
                    SELECT id, name, document_type, version, status,
                           created_at, updated_at
                    FROM documents
                    WHERE id = %s
                    """,
                    (document_id,),
                )
            return cur.fetchone()
```

The exact production implementation can later introduce dependency injection, pooling, typed models, migrations, observability, and more sophisticated error handling.

41. PostgreSQL vs Python Dictionary

- Dictionary: fast local access, but process-local and non-persistent.
- PostgreSQL: persistent, queryable, transactional, concurrent, constraint-aware, and designed for durable application data.

42. PostgreSQL vs SQLite

SQLite is an embedded database stored in a file and is excellent for many small applications, local tools, and tests. PostgreSQL is a full client-server relational database system designed for larger multi-user workloads and production backend systems.

For Project 1, PostgreSQL is the target because the project is intended to demonstrate production backend engineering.

43. PostgreSQL vs NoSQL

A relational database is particularly useful when the application has structured entities, relationships, constraints, transactions, and complex queries.

NoSQL databases can be valuable when different access patterns, document models, or scale characteristics make them a better fit.

The engineering skill is not memorizing that one database is always better; it is understanding workload, consistency, query patterns, operational requirements, and trade-offs.

44. PostgreSQL + Redis

FastAPI
■■■ PostgreSQL → durable application state
■■■ Redis → cache / temporary state / coordination

PostgreSQL is the durable system of record. Redis can provide fast in-memory caching, ephemeral state, rate limiting, queues, and other low-latency use cases.

45. PostgreSQL + Vector Search

Project 1 will eventually require vector retrieval for RAG. PostgreSQL can be extended with vector capabilities such as pgvector.

This allows structured relational metadata and vector representations to be used together, which can be useful for document retrieval systems.

The learning progression should be: first understand relational persistence and SQL, then add vector search and retrieval engineering.

46. SQLAlchemy and ORMs

SQLAlchemy is a Python database toolkit and ORM. An ORM maps Python objects and application models to relational database concepts.

ORMs can improve developer productivity, but they should not replace understanding SQL.

For this preparation path, learn SQL and PostgreSQL fundamentals first. Then learn SQLAlchemy so you understand what abstraction it provides and when to use it.

Interviewers may expect you to understand the SQL generated by an ORM, indexes, transactions, joins, connection pooling, and query performance.

47. Error Handling

- Connection failure.
- Authentication failure.
- Timeout.
- Constraint violation.
- Serialization or transaction conflict.
- Deadlock.
- Network failure.
- Database unavailable.
- Invalid SQL.
- Unexpected data.

Production code should distinguish expected application errors from infrastructure failures and expose safe, meaningful API responses.

48. Concurrency

A production API can receive many requests simultaneously. Multiple requests may read or modify the same records.

PostgreSQL provides transaction isolation and concurrency-control mechanisms. Application engineers must understand what happens when concurrent operations touch the same data.

Interview-level topics include transaction isolation, locks, deadlocks, race conditions, optimistic versus pessimistic approaches, and idempotency.

49. Database Reliability

- Connection timeouts.
- Query timeouts.
- Connection pool limits.
- Retries only where safe.
- Transaction boundaries.
- Idempotency.
- Backups and recovery.
- Monitoring and alerting.
- Capacity planning.
- Failure handling.

A production database is part of a distributed system. Your application must handle the possibility that the database is slow, unavailable, or partially failing.

50. Security

- Never hard-code passwords in source code.
- Use environment/configuration management for secrets.
- Use least-privilege database users.
- Use parameterized queries.
- Restrict network access.
- Encrypt traffic where appropriate.
- Protect backups.
- Audit sensitive operations.
- Avoid exposing raw database errors to API clients.

51. Observability

A production PostgreSQL-backed service should make database behavior observable.

Useful signals include request latency, query latency, errors, connection-pool utilization, throughput, and resource utilization.

For Project 1, this connects directly to the larger observability stack: OpenTelemetry, Prometheus, Grafana, structured logging, and distributed tracing.

52. Project 1 End-to-End Example

```
POST /documents
↓
Validate request
↓
Document Service
↓
Generate document ID
↓
Document Repository
↓
INSERT INTO documents
↓
PostgreSQL
↓
Return API response

UPDATE documents
SET status = 'processed',
    updated_at = NOW()
WHERE id = %s;

SELECT id, name, document_type, version, status,
       created_at, updated_at
FROM documents
WHERE id = %s;
```

53. Immediate Project Implementation Sequence

- 1. Install PostgreSQL.
- 2. Create the ai_engineering database.
- 3. Create the documents table.
- 4. Install Psycopg.
- 5. Configure DATABASE_URL.
- 6. Create the database connection layer.
- 7. Replace the dictionary repository.
- 8. Implement INSERT.
- 9. Implement SELECT.
- 10. Implement UPDATE.
- 11. Implement DELETE.
- 12. Test persistence.
- 13. Test database failures and invalid operations.
- 14. Document database setup.
- 15. Commit and push the implementation.

Key milestone: FastAPI should continue to communicate through the DocumentService → DocumentRepository abstraction, while the repository persists data to PostgreSQL instead of a Python dictionary.

54. What You Should Understand Before Moving On

- What PostgreSQL is and why it is used.
- How a Python application connects to PostgreSQL.
- What a database, schema, table, row, and column are.

- How to create and inspect tables.
- How PostgreSQL data types work.
- How primary keys and constraints protect data.
- How INSERT, SELECT, UPDATE, and DELETE work.
- How WHERE, ORDER BY, LIMIT, GROUP BY, and HAVING work.
- How JOINS connect relational data.
- What transactions and ACID mean.
- How commit and rollback work in Python.
- Why parameterized queries are necessary.
- Why connection pooling matters.
- Why migrations matter.
- Why indexes help and why they have costs.
- How EXPLAIN helps diagnose query performance.
- How PostgreSQL fits into FastAPI architecture.
- How PostgreSQL should be tested.
- How PostgreSQL fits with Redis and future vector search.

55. Interview Questions

What is PostgreSQL?

PostgreSQL is an open-source relational database management system that provides SQL querying, transactions, constraints, indexing, concurrency control, and durable persistent storage.

What is the difference between SQL and PostgreSQL?

SQL is a database query language and standard. PostgreSQL is a database management system that implements SQL and provides additional database capabilities.

Why use PostgreSQL instead of a Python dictionary?

A dictionary is process-local and non-persistent. PostgreSQL provides durable storage, transactions, concurrency, constraints, indexing, and powerful queries.

What is a primary key?

A primary key uniquely identifies a row in a table and enforces uniqueness and non-null identity semantics.

What is a foreign key?

A foreign key references a key in another table and is used to enforce referential integrity.

What is NULL?

NULL represents missing or unknown information. It is not equivalent to zero, false, or an empty string.

What is a transaction?

A transaction is a logical unit of database work whose operations are committed or rolled back according to transaction semantics.

What does ACID mean?

Atomicity, Consistency, Isolation, and Durability.

Why use parameterized queries?

They separate SQL structure from values and protect against SQL injection when handling untrusted input.

What is an index?

An index is a data structure that can accelerate selected query patterns, at the cost of additional storage and write/maintenance overhead.

Why not create an index on every column?

Indexes consume resources and must be maintained during writes. Excessive indexes can slow INSERT, UPDATE, and DELETE operations.

What is connection pooling?

Connection pooling reuses a controlled set of database connections instead of creating a new connection for every request.

What is a migration?

A migration is a versioned, repeatable schema change used to evolve a database safely over time.

What is JOIN?

JOIN combines rows from related tables using a specified relationship.

What is the repository pattern?

It isolates persistence operations from business and HTTP layers, keeping database access organized and testable.

What is EXPLAIN?

EXPLAIN shows the execution plan PostgreSQL intends to use for a query.

What is EXPLAIN ANALYZE?

EXPLAIN ANALYZE executes the query and reports actual execution information, helping diagnose performance.

DELETE vs TRUNCATE vs DROP?

DELETE removes selected rows and can use WHERE; TRUNCATE removes all rows efficiently; DROP removes the database object itself.

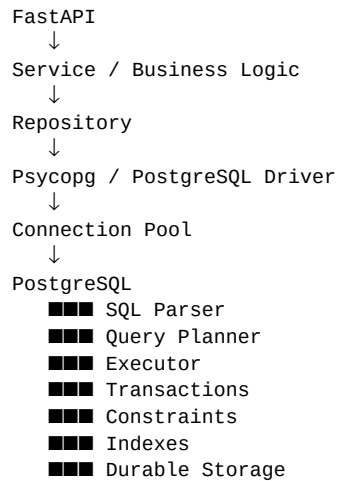
Why is PostgreSQL important for a FastAPI application?

It provides durable, transactional, concurrent persistent storage behind the API and allows application state to survive process restarts.

How would you test database code?

Use a separate test database, controlled fixtures, isolation/cleanup, and tests for success, missing data, constraints, transactions, and failures.

56. Final Mental Model



For the Google-level engineering profile being built, PostgreSQL is not just a technology to memorize. It should become part of a working backend system. The goal is to understand SQL, implement persistence in Project 1, test it, measure it, handle failures, and later connect it to caching, asynchronous processing, RAG, observability, Docker, Kubernetes, and GCP.